

Practico 02 – Reducciones

Computabilidad y Complejidad (Teoría de la Computación II)
FCEFQyN – Universidad Nacional de Río Cuarto
Primer Cuatrimestre de 2026

Resolución completa del Practico 02. Apoyada en el Capítulo 5 del libro de M. Sipser, *Introduction to the Theory of Computation*, y en el material teórico de la cátedra ([Reducibilidad.pdf](#)).

Índice

Introducción	1
1. Ejercicio 1: lectura del Capítulo 5	2
2. Ejercicio 2: movimiento del cabezal en el extremo izquierdo	2
3. Ejercicio 3: lenguajes cerrados bajo reverso	4
4. Ejercicio 4: estados inalcanzables (código muerto)	5
5. Ejercicio 5: Problema de Correspondencia de Post	7
6. Ejercicio 6: transitividad de $\leq_m$	8
7. Ejercicio 7: $\leq_m$ y regularidad	9
8. Ejercicio 8: A reconocible y $A \leq_m \bar{A}$ implican A decidible	10
9. Ejercicio 9: A es reconocible $\iff A \leq_m A_{TM}$	11
10. Ejercicio 10: indecidibilidad de <i>All</i> por Rice	12
Conclusiones	13

Introducción

El presente trabajo resuelve la totalidad de los ejercicios del Práctico 02 de la asignatura, centrado en el método de *reducciones* para probar indecidibilidad y para clasificar problemas de acuerdo a su reconocibilidad. La guía teórica utilizada es:

- el Capítulo 5 del libro de Sipser ([chapter-05.pdf](#)), que cubre las nociones de *reducibility*, *computation histories*, *Post Correspondence Problem* y *mapping reducibility*;
- las láminas de la cátedra ([Reducibilidad.pdf](#)), que sintetizan la estrategia general de reducción, el Teorema de Rice y la noción formal de función computable.

Convenciones de notación. A lo largo del documento adoptamos las siguientes convenciones, en línea con Sipser:

- Σ es un alfabeto y Σ^* el conjunto de cadenas finitas sobre Σ .
- $\langle M \rangle$ denota una codificación de la Máquina de Turing M , y $\langle M, w \rangle$ una codificación del par (M, w) .
- $\mathcal{L}(M)$ es el lenguaje reconocido por M .
- $\bar{A} = \Sigma^* \setminus A$ es el complemento de A .
- $A \leq_m B$ significa que existe una función computable $f : \Sigma^* \rightarrow \Sigma^*$ con $x \in A \iff f(x) \in B$ (reducibilidad many-one).
- “Decidible” = Turing-decidible; “reconocible” = Turing-reconocible.

Lenguajes de referencia. Los siguientes lenguajes se asumen *indecidibles* (resultados de Sipser 4.11, 5.1 y 5.2):

$$\begin{aligned} A_{\text{TM}} &= \{ \langle M, w \rangle \mid M \text{ es una MT que acepta } w \}, \\ \text{HALT}_{\text{TM}} &= \{ \langle M, w \rangle \mid M \text{ se detiene con entrada } w \}, \\ E_{\text{TM}} &= \{ \langle M \rangle \mid \mathcal{L}(M) = \emptyset \}. \end{aligned}$$

Además, la herramienta básica que utilizaremos repetidamente es el *Corolario 5.23* de Sipser: *si $A \leq_m B$ y A es indecible, entonces B es indecible*, y la versión análoga para reconocibilidad (*Teorema 5.28*). Y, para el Ejercicio 10, el *Teorema de Rice*.

1. Ejercicio 1: lectura del Capítulo 5

El primer ítem del práctico consiste en *leer el Capítulo 5 del libro de Sipser*. Se asume realizado como prerrequisito teórico para el resto de los ejercicios. Las nociones que se utilizan a partir del Ejercicio 2 incluyen, entre otras:

- la indecibilidad de A_{TM} (Teorema 4.11);
- la reducibilidad informal usada para probar la indecibilidad de HALT_{TM} , E_{TM} , $\text{REGULAR}_{\text{TM}}$ y EQ_{TM} (Teoremas 5.1–5.4);
- la técnica de *computation histories* usada para los autómatas linealmente acotados (Sección 5.1, Teorema 5.10);
- el Problema de Correspondencia de Post y su indecibilidad (Sección 5.2, Teorema 5.15);
- la noción formal de *mapping reducibility* (Sección 5.3, Definición 5.20) y los Teoremas 5.22, 5.28 y el Corolario 5.23;
- el *Teorema de Rice* (Problema 5.28).

Cada ejercicio que sigue cita explícitamente las definiciones y resultados del capítulo que invoca.

2. Ejercicio 2: movimiento del cabezal en el extremo izquierdo

Enunciado

Considere el problema de determinar si una MT, con la entrada w , alguna vez intenta mover el cabezal a la izquierda estando en la posición más a la izquierda de la cinta. Demostrar que este problema es indecible.

Formalización

Definición 2.1. Definimos el lenguaje

$LM = \{\langle M, w \rangle \mid M \text{ intenta, durante su ejecución con entrada } w, \text{ mover el cabezal a la izquierda desde la celda } 0\}$

Teorema 2.2. LM es indecidible.

Idea de la demostración

Reducimos A_{TM} a LM . La estrategia es construir, a partir de $\langle M, w \rangle$, una MT M' que se comporta como M pero con dos modificaciones:

1. antes de simular a M , M' marca la celda más a la izquierda con un símbolo especial \$, de modo de poder distinguirla siempre, y trabaja a partir de la celda inmediatamente a la derecha;
2. durante la simulación, M' *nunca* cruza el marcador hacia la izquierda; pero *si y solo si* la simulación alcanza el estado de aceptación de M , M' se desplaza al símbolo \$ y ejecuta una transición que mueve el cabezal a la izquierda estando sobre el extremo.

Así, M' intenta mover el cabezal a la izquierda desde el extremo izquierdo si y solo si M acepta w .

Construcción formal

La reducción f envía $\langle M, w \rangle$ a $\langle M', w \rangle$, donde M' es la MT que sigue:

$M' = \text{“Con entrada } x\text{:”}$

1. Correr una posición a la derecha el contenido de la cinta y escribir el símbolo especial \$ en la celda 0.
2. Posicionar el cabezal en la celda 1 (la primera celda a la derecha de \$).
3. Simular a M sobre w (la cadena fija cableada en la descripción de M'). En cada paso, si la transición de M indicaría mover el cabezal a la izquierda y la celda actual contiene \$, M' se queda en su lugar (no cruza el marcador).
4. Si la simulación entra en q_{accept} de M , M' se desplaza a la izquierda hasta encontrar \$ y, desde allí, ejecuta una transición que indica mover el cabezal a la izquierda.
5. Si la simulación entra en q_{reject} de M , M' se detiene sin haber intentado nunca cruzar el extremo izquierdo.”

Notar que M' ignora x : siempre simula a M con la cadena fija w , cableada en la descripción.

Computabilidad de la reducción

La función $f(\langle M, w \rangle) = \langle M', w \rangle$ es computable. En efecto, una MT puede mecánicamente:

- agregar a $\langle M \rangle$ los estados que implementan los pasos 1 y 2;
- modificar las transiciones de $\langle M \rangle$ que mueven a la izquierda para que respeten el marcador \$;
- agregar las transiciones que implementan los pasos 4 y 5.

Esto encaja en la Definición 5.17 de Sipser.

Correctitud

Demostración. Mostramos $\langle M, w \rangle \in A_{\text{TM}} \iff \langle M', w \rangle \in LM$.

($\Rightarrow$) Si M acepta w , la simulación del paso 3 eventualmente entra en q_{accept} de M . El paso 4 entonces lleva el cabezal hasta $\$$ y desde allí intenta moverse a la izquierda. Por construcción, ese intento ocurre estando M' en la celda más a la izquierda. Luego $\langle M', w \rangle \in LM$.

($\Leftarrow$) Supongamos $\langle M', w \rangle \in LM$. Por construcción, los únicos puntos en los que M' podría intentar moverse a la izquierda desde el extremo son:

- dentro de la simulación del paso 3, lo cual fue *descartado* por la modificación que reemplaza esos movimientos por “quedarse en el lugar”;
- luego del paso 4, que solo se ejecuta si la simulación alcanzó q_{accept} .

Entonces, si M' intentó mover el cabezal a la izquierda desde el extremo, la simulación alcanzó q_{accept} , es decir, M acepta w . Luego $\langle M, w \rangle \in A_{\text{TM}}$.

Tenemos $A_{\text{TM}} \leq_m LM$. Como A_{TM} es indecidible, por el Corolario 5.23 de Sipser, LM es indecidible. $\square$

3. Ejercicio 3: lenguajes cerrados bajo reverso

Enunciado

Sea $T = \{\langle M \rangle \mid M \text{ es una MT que acepta } w^{\mathcal{R}} \text{ si esta acepta } w\}$. Mostrar que T es indecidible.

Interpretación

La condición “ M acepta $w^{\mathcal{R}}$ si acepta w ” se entiende para *toda* cadena w . Así T es el conjunto de descripciones de MT cuyo lenguaje es *cerrado bajo reverso*:

$$T = \{\langle M \rangle \mid \forall w \in \Sigma^* : w \in \mathcal{L}(M) \Rightarrow w^{\mathcal{R}} \in \mathcal{L}(M)\}.$$

Es importante notar que T depende solo de $\mathcal{L}(M)$ y no de la sintaxis o estructura interna de M . Esto convierte a T en una propiedad *semántica* de lenguajes, lo que permitirá también una demostración alternativa por Rice (véase al final de la sección).

Estrategia

Daremos una reducción directa $A_{\text{TM}} \leq_m T$. La idea: a partir de $\langle M, w \rangle$ construimos una MT M_w cuyo lenguaje es

- $\{01, 10\}$ si M acepta w (cerrado bajo reverso);
- $\{01\}$ si M no acepta w (*no* cerrado bajo reverso, pues $10 = 01^{\mathcal{R}} \notin \{01\}$).

Construcción de M_w

$M_w =$ “**Con entrada x :**

1. Si $x = 01$, aceptar.
2. Si $x = 10$, simular a M sobre w . Si M acepta, aceptar; si M rechaza, rechazar.
3. Si x es cualquier otra cadena, rechazar.”

La descripción $\langle M_w \rangle$ se construye algorítmicamente a partir de $\langle M, w \rangle$: alcanza con agregar a M unos pocos estados de comparación con las cadenas 01 y 10 y un selector que en el caso 10 deriva al subprograma que simula a M con w . Por lo tanto la función $f(\langle M, w \rangle) = \langle M_w \rangle$ es computable.

Correctitud

Demostración. Verifiquemos que $\langle M, w \rangle \in A_{\text{TM}} \iff \langle M_w \rangle \in T$.

($\Rightarrow$) Si M acepta w , entonces:

- M_w acepta 01 por el paso 1.
- M_w acepta 10 por el paso 2 (la simulación de M sobre w acepta).
- M_w rechaza cualquier otra cadena por el paso 3.

Así $\mathcal{L}(M_w) = \{01, 10\}$, que es cerrado bajo reverso pues $01^{\mathcal{R}} = 10 \in \mathcal{L}(M_w)$ y $10^{\mathcal{R}} = 01 \in \mathcal{L}(M_w)$. Luego $\langle M_w \rangle \in T$.

($\Leftarrow$) Si M no acepta w (rechaza o cicla), entonces:

- M_w acepta 01 por el paso 1.
- M_w no acepta 10: la simulación de M sobre w no acepta, y por lo tanto M_w no entra en q_{accept} con entrada 10 (rechaza si M rechaza, cicla si M cicla).
- Cualquier otra cadena se rechaza por el paso 3.

Entonces $\mathcal{L}(M_w) = \{01\}$, que *no* es cerrado bajo reverso, ya que requeriría $01^{\mathcal{R}} = 10 \in \mathcal{L}(M_w)$ y no se cumple. Luego $\langle M_w \rangle \notin T$.

Tenemos $A_{\text{TM}} \leq_m T$, y por el Corolario 5.23 de Sipser, T es indecidible. $\square$

Demostración alternativa por el Teorema de Rice

Observación. Una vía más breve es invocar el Teorema de Rice. La propiedad “ $\mathcal{L}(M)$ es cerrado bajo reverso” es:

- *semántica*: depende solo de $\mathcal{L}(M)$;
- *no trivial*: la satisface la MT con $\mathcal{L}(M) = \emptyset$ (cerrado vacuamente bajo reverso) y no la satisface una MT con $\mathcal{L}(M) = \{01\}$.

Por Rice, T es indecidible.

4. Ejercicio 4: estados inalcanzables (código muerto)

Enunciado

Un estado de una MT se dice *inalcanzable* cuando no se puede llegar a él con ningún input. Demostrar que el problema de ver si una MT tiene un estado inalcanzable es indecidible. (En la práctica, la detección de *código muerto*.)

Formalización

Definición 4.1. Un estado q de la MT M es *inalcanzable* si no existe entrada $x \in \Sigma^*$ tal que la ejecución de M sobre x pase por q . Definimos

$$UNREACH = \{\langle M \rangle \mid M \text{ tiene al menos un estado inalcanzable}\}.$$

Teorema 4.2. *UNREACH es indecidible.*

Estrategia

Reduciremos $\overline{A_{TM}}$ a *UNREACH*. Equivalentemente, daremos una reducción tal que “ M acepta w ” se corresponde con “todos los estados de M' son alcanzables”. Como A_{TM} es indecidible y la decidibilidad se preserva bajo complemento, esto basta para concluir que *UNREACH* es indecidible.

A partir de $\langle M, w \rangle$ construimos una MT M' con un estado distinguido q_* (*testigo*) que será alcanzable solamente si M acepta w . El resto de los estados de M' será alcanzable en cualquier caso, por medio de un *tour inicial* sobre todos esos estados desencadenado por una entrada auxiliar.

Construcción de M'

Los estados de M' son: el inicial q_0 ; un rechazo q_R ; un aceptación q_A ; el testigo q_* ; y los estados auxiliares q_{sim} usados para simular a M sobre w .

$M' =$ “**Con entrada x :**

1. Caso $x = 0$: ejecutar una secuencia finita de movimientos triviales que recorre todos los estados de M' salvo q_* . Al terminar, ir a q_R y rechazar.
2. Caso $x = 1$: simular a M sobre w :
 - si M acepta, transitar a q_* y luego a q_A ;
 - si M rechaza, ir a q_R .
3. Cualquier otra entrada: ir a q_R .”

El *tour* del paso 1 es siempre realizable: en una MT cualquiera se puede agregar una rutina inicial finita que recorre cada uno de sus estados una sola vez antes de continuar con el comportamiento principal. La función $f(\langle M, w \rangle) = \langle M' \rangle$ es computable, ya que toda esta transformación se realiza algorítmicamente sobre $\langle M \rangle$ y w .

Correctitud

Demostración. Mostramos $\langle M, w \rangle \in A_{TM} \iff \langle M' \rangle \notin UNREACH$.

($\Rightarrow$) Si M acepta w :

- Sobre la entrada 0, M' recorre todos los estados salvo q_* gracias al *tour* del paso 1.
- Sobre la entrada 1, la simulación llega a q_{accept} de M , por lo que M' transita a q_* . Luego q_* es alcanzable.

Concluimos que todos los estados de M' son alcanzables y por lo tanto $\langle M' \rangle \notin UNREACH$.

($\Leftarrow$) Si M no acepta w (rechaza o cicla):

- En cualquier entrada distinta de 1, M' no entra en la rama del paso 2 y no pasa por q_* .
- En la entrada 1, la simulación de M no alcanza q_{accept} , por lo que M' no transita a q_* .

q_* resulta inalcanzable, así que $\langle M' \rangle \in UNREACH$.

Hemos exhibido $\overline{A_{TM}} \leq_m UNREACH$. Como $\overline{A_{TM}}$ es indecidible (un lenguaje es decidible si y solo si lo es su complemento), por el Corolario 5.23 de Sipser, *UNREACH* es indecidible. $\square$

Interpretación

Observacion. El resultado se lee, en la práctica del software, como: *el problema de detectar código muerto en una MT (estados a los que ningún flujo de entrada llega) es indecidible*. Por eso las herramientas reales de análisis estático utilizan aproximaciones conservadoras: sobre el modelo general de cómputo, la pregunta exacta no admite algoritmo.

5. Ejercicio 5: Problema de Correspondencia de Post

Enunciado

Encontrar un *matching* para la siguiente instancia del Problema de Post:

$$\left\{ \frac{ab}{abab}, \frac{b}{a}, \frac{aba}{b}, \frac{aa}{a} \right\}.$$

Recordatorio teórico

Una instancia del Problema de Correspondencia de Post (PCP) es una colección finita de fichas

$$P = \left\{ \left[\frac{t_1}{b_1} \right], \left[\frac{t_2}{b_2} \right], \dots, \left[\frac{t_k}{b_k} \right] \right\}.$$

Un *matching* es una secuencia de índices $i_1, i_2, \dots, i_l$ (con repeticiones permitidas) tal que

$$t_{i_1} t_{i_2} \dots t_{i_l} = b_{i_1} b_{i_2} \dots b_{i_l}.$$

Numeración

Numeramos las fichas dadas:

Índice i	Arriba (t_i)	Abajo (b_i)
1	ab	$abab$
2	b	a
3	aba	b
4	aa	a

Análisis preliminar

- La ficha 1 contribuye dos símbolos arriba y cuatro abajo: deja un “déficit” de longitud arriba que debe compensarse con fichas donde haya *más* símbolos arriba que abajo.
- Las candidatas para compensar son la ficha 3 ($|t_3| = 3 > |b_3| = 1$) y la ficha 4 ($|t_4| = 2 > |b_4| = 1$).
- En cualquier matching, la última ficha debe terminar en el mismo símbolo arriba y abajo.

Solución

Probamos la secuencia de índices $(4, 4, 2, 1)$.

Verificación. Concatenando arriba:

$$t_4 t_4 t_2 t_1 = aa \cdot aa \cdot b \cdot ab = aaaabab.$$

Concatenando abajo:

$$b_4 b_4 b_2 b_1 = a \cdot a \cdot a \cdot abab = aaaabab.$$

Ambas cadenas coinciden:

$$aaaabab = aaaabab.$$

Por lo tanto $(4, 4, 2, 1)$ es un matching válido para la instancia.

Visualización. Las fichas se acomodan así:

$$\left[\frac{aa}{a} \right] \left[\frac{aa}{a} \right] \left[\frac{b}{a} \right] \left[\frac{ab}{abab} \right]$$

y produce, leyendo arriba y abajo,

$$\begin{array}{c|c|c|c|c|c|c} a & a & a & a & b & a & b \\ \hline a & a & a & a & b & a & b \end{array}$$

$(4, 4, 2, 1)$ es un matching de la instancia, con cadena común $aaaabab$.

6. Ejercicio 6: transitividad de $\leq_m$

Enunciado

Demostrar que $\leq_m$ es transitiva.

Proposición 6.1. Si $A \leq_m B$ y $B \leq_m C$, entonces $A \leq_m C$.

Demostración. Por hipótesis disponemos de:

1. $f : \Sigma^* \rightarrow \Sigma^*$ computable, con $x \in A \iff f(x) \in B$ para todo x ;
2. $g : \Sigma^* \rightarrow \Sigma^*$ computable, con $y \in B \iff g(y) \in C$ para todo y .

Definimos $h : \Sigma^* \rightarrow \Sigma^*$ como la composición:

$$h(x) = g(f(x)).$$

h es computable. Sean M_f y M_g MT que computan f y g respectivamente. Construimos M_h :

$M_h =$ “**Con entrada x :**

1. Simular M_f sobre x hasta que termine. La cinta queda con $f(x)$.
2. Simular M_g sobre el contenido actual de la cinta. Al terminar contiene $g(f(x)) = h(x)$.
3. Detenerse.”

Como M_f y M_g terminan en cualquier entrada (Definición 5.17 de Sipser), M_h también termina y deja $h(x)$ en la cinta. Luego h es computable.

h es una reducción. Para todo $x \in \Sigma^*$:

$$x \in A \stackrel{(1)}{\iff} f(x) \in B \stackrel{(2)}{\iff} g(f(x)) \in C \iff h(x) \in C.$$

La equivalencia (1) es la hipótesis sobre f aplicada a x ; la (2) es la hipótesis sobre g aplicada a $f(x)$.

Concluimos $A \leq_m C$. □

Observación. Esta propiedad es un atajo poderoso: una vez probado por ejemplo $A_{\text{TM}} \leq_m \text{HALT}_{\text{TM}}$ y $\text{HALT}_{\text{TM}} \leq_m E_{\text{TM}}$, la transitividad da $A_{\text{TM}} \leq_m E_{\text{TM}}$ sin construir una nueva reducción. La cadena $A_{\text{TM}} \leq_m \text{MPCP} \leq_m \text{PCP}$ del Capítulo 5 de Sipser implica entonces $A_{\text{TM}} \leq_m \text{PCP}$, y por ende la indecidibilidad de PCP.

7. Ejercicio 7: $\leq_m$ y regularidad

Enunciado

Decir si se cumple o no la siguiente afirmación: si $A \leq_m B$ y B es un lenguaje regular, esto implica que A es regular. Fundamental.

Respuesta

La afirmación es falsa en general.

La intuición: la relación $\leq_m$ preserva *decidibilidad* y *reconocibilidad* (Teoremas 5.22 y 5.28 de Sipser), pero no preserva propiedades más finas como la regularidad. La razón es que la potencia de cómputo disponible para definir la reducción f es la de una MT general, no la de un autómata finito. Esa MT puede ocultar tareas no regulares dentro de una clasificación binaria de cadenas.

Contraejemplo

Consideremos

$$A = \{0^n 1^n \mid n \geq 0\}, \quad B = \{1\}.$$

- B es regular: es un singleton, reconocido por un AFD trivial.
- A no es regular: clásico, por el lema de bombeo.

Construcción de f . Como A es decidible, existe un algoritmo que decide pertenencia a A . Definimos

$$f(x) = \begin{cases} 1 & \text{si } x \in A, \\ 0 & \text{si } x \notin A. \end{cases}$$

f es computable: una MT simula el decisor de A , borra la cinta y escribe 1 o 0 según la respuesta.

f es una reducción. Para todo x :

$$x \in A \iff f(x) = 1 \iff f(x) \in B.$$

Por lo tanto $A \leq_m B$. Pero B es regular y A no lo es.

Por qué falla la afirmación

La regularidad es una propiedad estructural de los lenguajes (caracterizada por autómatas finitos, expresiones regulares, gramáticas regulares, etc.). La relación $\leq_m$ usa funciones computables arbitrarias y solo controla la *pertenencia* mediante una función potencialmente compleja, sin imponer restricciones de complejidad o estructura sobre el lenguaje origen.

Observacion. Las propiedades que sí se preservan bajo $\leq_m$ son las que se “comportan bien” frente a máquinas potentes:

- Decidibilidad (Teorema 5.22): si $A \leq_m B$ y B es decidable, entonces A es decidable.
 - Reconocibilidad (Teorema 5.28): si $A \leq_m B$ y B es reconocible, entonces A es reconocible.
-

8. Ejercicio 8: A reconocible y $A \leq_m \bar{A}$ implican A decidable

Enunciado

Demostrar que si A es reconocible y $A \leq_m \bar{A}$, entonces A es decidable.

Proposición 8.1. *Si A es Turing-reconocible y $A \leq_m \bar{A}$, entonces A es decidable.*

Idea

Combinaremos dos resultados de Sipser:

- Teorema 4.22: A es decidable si y solo si A y $\bar{A}$ son ambos reconocibles.
- Teorema 5.28: si $X \leq_m Y$ y Y es reconocible, entonces X es reconocible.

Como A ya es reconocible por hipótesis, basta mostrar que también $\bar{A}$ es reconocible. Para ello observamos que la reducción $A \leq_m \bar{A}$ implica la simétrica $\bar{A} \leq_m A$ con la *misma* función f .

Demostración

Demostración. **Paso 1:** $A \leq_m \bar{A}$ implica $\bar{A} \leq_m A$ con la misma f .

Sea f computable con $x \in A \iff f(x) \in \bar{A}$ para todo x . Negando ambos lados de la equivalencia:

$$x \notin A \iff f(x) \notin \bar{A}.$$

Equivalentemente, $x \in \bar{A} \iff f(x) \in A$. Así la misma f reduce $\bar{A}$ a A , esto es, $\bar{A} \leq_m A$.

Paso 2: $\bar{A}$ es reconocible.

Como A es reconocible y $\bar{A} \leq_m A$, el Teorema 5.28 implica que $\bar{A}$ es reconocible.

Concretamente, si M_A reconoce A , un reconocedor de $\bar{A}$ es:

$R_{\bar{A}} =$ “Con entrada x :

1. Computar $f(x)$.
2. Simular M_A sobre $f(x)$. Si acepta, aceptar.”

Paso 3: A es decidable.

Tenemos que A y $\bar{A}$ son ambos reconocibles. Por el Teorema 4.22 de Sipser, A es decidable. $\square$

Observacion. El *contrapositivo* es informativo: si A es reconocible pero no decidible (por ejemplo, $A = \overline{A_{\text{TM}}}$), entonces necesariamente $A \not\leq_m \overline{A}$. Esto muestra (Sipser, Ejercicio 5.5) que $A_{\text{TM}} \not\leq_m \overline{A_{\text{TM}}}$.

9. Ejercicio 9: A es reconocible $\iff A \leq_m A_{\text{TM}}$

Enunciado

Demostrar que A es reconocible si y solo si $A \leq_m A_{\text{TM}}$.

Proposición 9.1. Para todo $A \subseteq \Sigma^*$,

$$A \text{ es Turing-reconocible} \iff A \leq_m A_{\text{TM}}.$$

Demostración. Probamos las dos direcciones.

($\Rightarrow$) Sea A reconocible y M_A una MT con $\mathcal{L}(M_A) = A$. Definimos

$$f : \Sigma^* \rightarrow \Sigma^*, \quad f(x) = \langle M_A, x \rangle.$$

f es computable. Una MT puede:

1. leer x de la cinta;
2. generar la descripción fija $\langle M_A \rangle$ (cableada en la propia descripción de la MT que computa f);
3. construir la concatenación $\langle M_A, x \rangle$ con un separador.

La operación es algorítmica y termina siempre, por lo que f es computable (Definición 5.17 de Sipser).

f es una reducción de A a A_{TM} . Para todo x :

$$x \in A \iff M_A \text{ acepta } x \iff \langle M_A, x \rangle \in A_{\text{TM}} \iff f(x) \in A_{\text{TM}}.$$

Concluimos $A \leq_m A_{\text{TM}}$.

($\Leftarrow$) Supongamos $A \leq_m A_{\text{TM}}$ con reducción f . A_{TM} es reconocible: la máquina universal U acepta $\langle M, w \rangle$ ssi M acepta w (Teorema 4.11 y la noción clásica de UTM en Sipser).

Construimos un reconocedor R para A :

$R =$ “Con entrada x :

1. Computar $f(x)$.
2. Simular U sobre $f(x)$. Si acepta, aceptar.”

Correctitud.

- Si $x \in A$, entonces $f(x) \in A_{\text{TM}}$ y U acepta $f(x)$. Luego R acepta.
- Si $x \notin A$, entonces $f(x) \notin A_{\text{TM}}$ y U no acepta $f(x)$ (rechaza o cicla). Luego R no acepta.

Así $\mathcal{L}(R) = A$, y A es reconocible. □

Observacion. Este resultado dice que A_{TM} es *completo* para la clase de lenguajes Turing-reconocibles bajo $\leq_m$: todo lenguaje reconocible se reduce a él. En particular, cualquier problema reconocible al cual A_{TM} se reduce hereda su indecibilidad.

10. Ejercicio 10: indecibilidad de All por Rice

Enunciado

Usar el Teorema de Rice para demostrar la indecibilidad de

$$All = \{\langle M \rangle \mid \mathcal{L}(M) = \Sigma^*\}.$$

(El enunciado del práctico contiene una errata menor: escribe $\mathcal{M} = \Sigma^*$, donde claramente se quiere decir $\mathcal{L}(M) = \Sigma^*$.)

Recordatorio: Teorema de Rice

Teorema 10.1 (Rice). Sea $P \subseteq \{\langle M \rangle \mid M \text{ es una MT}\}$. Si P cumple:

1. Es una propiedad semántica: si $\mathcal{L}(M_1) = \mathcal{L}(M_2)$, entonces $\langle M_1 \rangle \in P \iff \langle M_2 \rangle \in P$.
2. Es no trivial: $P \neq \emptyset$ y $P \neq \{\langle M \rangle \mid M \text{ es MT}\}$.

Entonces P es indecidible.

La idea de la demostración (Sipser, Problema 5.28; también en [Reducibilidad.pdf](#)) es construir, dado un decisor hipotético M_P , un decisor de A_{TM} usando un *testigo* $T \in P$ y la MT M_w asociada a la entrada $\langle M, w \rangle$.

Aplicación a All

Asociamos a All la propiedad

$$P(L) \iff L = \Sigma^*.$$

Es decir, $\langle M \rangle \in All$ si y solo si $\mathcal{L}(M) = \Sigma^*$.

1. All es una propiedad semántica. Si $\mathcal{L}(M_1) = \mathcal{L}(M_2)$, entonces

$$\langle M_1 \rangle \in All \iff \mathcal{L}(M_1) = \Sigma^* \iff \mathcal{L}(M_2) = \Sigma^* \iff \langle M_2 \rangle \in All.$$

2. All es no trivial.

- Sea M_{Σ^*} la MT que en cualquier entrada va inmediatamente a q_{accept} . Entonces $\mathcal{L}(M_{\Sigma^*}) = \Sigma^*$ y $\langle M_{\Sigma^*} \rangle \in All$. Luego $All \neq \emptyset$.
- Sea $M_{\emptyset}$ la MT que rechaza inmediatamente toda entrada. Entonces $\mathcal{L}(M_{\emptyset}) = \emptyset \neq \Sigma^*$ y $\langle M_{\emptyset} \rangle \notin All$. Luego All no contiene a todas las descripciones de MT.

Conclusión. All cumple las dos hipótesis del Teorema de Rice, por lo que es indecidible.

$All = \{\langle M \rangle \mid \mathcal{L}(M) = \Sigma^*\} \text{ es indecidible.}$

Conexión con el capítulo y demostración alternativa

Observacion. All y E_{TM} son “opuestos” en cierto sentido: E_{TM} pregunta si $\mathcal{L}(M) = \emptyset$ y All si $\mathcal{L}(M) = \Sigma^*$. Sipser prueba la indecibilidad de E_{TM} directamente por reducción desde A_{TM} (Teorema 5.2). Aquí el Teorema de Rice nos ahorra la construcción explícita.

Una demostración directa por reducción también es posible. Dado $\langle M, w \rangle$, construimos M_w que ignora su entrada y simula M sobre w , aceptando cuando M acepta. Si M acepta w , $\mathcal{L}(M_w) = \Sigma^*$ y $\langle M_w \rangle \in All$; en otro caso $\mathcal{L}(M_w) \neq \Sigma^*$ y $\langle M_w \rangle \notin All$. Así $A_{TM} \leq_m All$, lo cual implica la indecibilidad de All sin invocar Rice. Pero la vía por Rice, como pedía el enunciado, es mucho más concisa.

Conclusiones

A lo largo del práctico hemos puesto en práctica la técnica central del Capítulo 5 de Sipser y de la teoría de la cátedra: la *reducción de problemas*.

- Los Ejercicios 2, 3 y 4 ilustran el uso de reducciones desde A_{TM} (o su complemento) para probar indecibilidad de propiedades concretas de las MT.
- El Ejercicio 5 muestra cómo, a pesar de que el PCP es indecible en general, instancias particulares pueden resolverse “a mano”.
- El Ejercicio 6 establece la transitividad de $\leq_m$, propiedad fundamental que permite encadenar reducciones.
- El Ejercicio 7 evidencia los límites de $\leq_m$: preserva decidibilidad y reconocibilidad pero no propiedades más estructurales como la regularidad.
- Los Ejercicios 8 y 9 muestran que $\leq_m$ se comporta de manera natural respecto del complemento y caracteriza la Turing-reconocibilidad mediante A_{TM} .
- El Ejercicio 10 aplica el Teorema de Rice como herramienta “meta” para concluir indecibilidad sin construir una reducción explícita.

En conjunto, los resultados consolidan la imagen de A_{TM} como problema *paradigmático* de la indecibilidad y de $\leq_m$ como la herramienta básica para clasificar problemas en torno a él.